

Take-home interview exercise: Equity signals, data integrity, and return prediction

Time budget

Please spend no more than a day in a weekend. If you do not have time to finish the following it is fine—you don't need to finish the entire pieces. We are more interested in your judgment, coding discipline, and understanding of financial data than in producing the most elaborate model.

We encourage you to use AI in the process. However, if you use AI in this process, please state clearly and faithfully which AI tool(s) that you have used and how much did you use AI in the process; in the process, we'd also like to see how you interact with AI to improve the outcome.

Data provided

You are given three SAS datasets:

1. `signals_raw_plus.sas7bdat`
Monthly stock-level signal values for a randomly selected set of U.S. stocks from 1995–2023/4. The key variables are PERMNO (a unique stock identifier) and fdate. The data manual is `Backtest_Manual_v2.pdf`.
2. `msf.sas7bdat`
CRSP (Center for Research in Security Prices)-style monthly stock-level information. The key variables are PERMNO and DATE (calendar month end) and RET (monthly stock return at month-end).
3. `factors_monthly.sas7bdat`
Monthly Fama-French/Carhart factor returns, including mktrf (excess market return), smb (SMB factor), hml (HML factor), umd (UMD factor), and rf (risk free rate).

Objective

Build a clean, reproducible pipeline to test whether stock-level signals predict future monthly stock returns. Your task is not to discover a publishable anomaly from this small random sample. Instead, we want to see whether you can correctly handle financial panel data, avoid look-ahead bias, form sensible portfolios, evaluate signal performance, and explain limitations.

Part 1. Data audit and merge design

Start by auditing the three datasets.

Report:

1. Number of observations and distinct PERMNOs in each dataset.
2. Date range of each dataset.

3. Whether the datasets contain all fields needed for a proper return-prediction test.
4. Whether there are duplicate PERMNO-month observations.
5. Which variables you would use as permanent identifiers and which you would avoid using as merge keys.

A strong answer should discuss why the data is an unbalanced panel, and why returns should be measured after signal formation.

Important: If any required variable is missing, clearly state what is missing and how you would fix the data before running the backtest.

Part 2. Signal selection and economic sign

Choose **four signals** from signals_raw_plus.sas7bdat, including at least one from each of the following categories:

1. Momentum or reversal
Examples: momentum, momentum12, ltr36, ltr60, seasonality.
2. Value
Examples: BtM, PtB, CF_P, Sale_P, PE.
3. Profitability or quality
Examples: GProf, OProf, ROA, ROE, PFS_Score.
4. Risk, distress, or limits to arbitrage
Examples: ivol, beta, chs, O_score, maxret.

For each signal, briefly explain:

1. What economic idea it is meant to capture.
2. Whether a higher value should predict higher or lower future returns.
3. Whether you need to multiply the signal by -1 so that “higher score” consistently means “higher expected return.”

For example, high momentum is usually expected to predict higher near-term returns, while high accruals, high asset growth, high idiosyncratic volatility, or high distress are often expected to predict lower future returns.

Part 3. Return-prediction dataset

Construct a monthly stock-level prediction dataset where signal values observed at the end of month t are used to predict stock returns in month $t+1$.

Required steps:

1. Convert all dates to a consistent monthly identifier.
2. Merge signal data to CRSP-style monthly return data by PERMNO and month.
3. Construct future one-month returns.
4. Merge factor returns by month.
5. Construct excess returns, $ret - rf$.
6. Document how you treat missing signals, missing returns, and extreme values.

Use only information available at or before portfolio formation month. Avoid look-ahead bias.

Part 4. Single-signal portfolio backtests

For each of your four selected signals:

1. Each month, sort stocks into quintiles based on the signal.
2. Calculate next-month equal-weighted returns for each quintile.
3. Construct a long-short portfolio:
 - Long the highest expected-return quintile.
 - Short the lowest expected-return quintile.
4. Report the following:
 - Average monthly long-short return.
 - Annualized volatility.
 - Sharpe ratio.
 - CAPM alpha.
 - Fama-French-Carhart four-factor alpha.
 - Maximum drawdown.
 - Average monthly number of stocks in each leg.

The WRDS Backtester forms portfolios using prior-month signal values and evaluates high-minus-low long-short returns, factor-adjusted returns, Sharpe ratio, drawdown, and turnover; use that general framework as your benchmark.

Part 5. Composite signal

Build one simple composite score using your five selected signals.

Suggested approach:

1. Winsorize each signal cross-sectionally by month.
2. Convert each signal into a monthly cross-sectional z-score or rank.
3. Align signal signs so that higher means better expected return.
4. Average the standardized signals into one composite score.
5. Repeat the quintile portfolio test using the composite score.

Discuss whether the composite improves performance relative to the individual signals.

Part 6. Short research memo

Submit a short memo, no more than **3 pages**, summarizing:

1. The data structure and any problems you found.
2. Your selected signals and their economic interpretation.
3. Your main return-prediction results.
4. Whether the results should be trusted, given the sample design.
5. What you would do next with the full CRSP/Compustat universe.
6. Any improvements you would make for an institutional-grade backtesting system.

Part 7. In addition to Part 6, any other thought in the process related to factor harvesting and portfolio risk management.

Deliverables

Submit:

1. A reproducible Python notebook or .py script.
2. Any helper functions or modules.
3. A short PDF memo.
4. A requirements.txt or environment note.
5. A brief README explaining how to run your code.

Python is preferred. SAS is acceptable for reading or preparing the SAS datasets, but the main analysis should be reproducible in Python.